

Workspace analysis

Marko Guberina

March 2, 2026

1 Setting

The following is a sub-problem of the cart-pulling problem, but with some easily generalizable principles. The task is to find the robot's workspace. In particular, the robot is dual armed, and has a rigid symmetric grasp of a cart's handlebar (which is also rigid). Since the robot and the are on flat terrain, and the grasp is rigid, the arm's workspace is effectively restricted to a plane parallel to the floor and the cart's height.

The robot is mobile, and it's task is to navigate the cart around. However, here it is enough to constrict it to path following. The claim is the following one. Fixing the distance between the robot and the cart (handlebar) does not change the steerability of the cart.

Definition of steerability Approximately, it is how large the difference in heading between the cart and the robot can be supported, combined with how close the two can be while supporting this heading.

Why this? The robot-cart system needs to follow paths. The larger the path curvature, the harder the path. To avoid pathological cases, and to be able to make concrete relevant statements, we need to specify the curve somehow. To make life simple, we choose a circle. A circle has constant curvature, which is $\frac{1}{r}$. Thus the question is what is the smallest circle that can be followed by the cart being held by the robot (the robot can follow any path exactly — if not, select the largest curvature the robot can follow). Thus, for the same amount of available heading, the closer the cart is to the robot, the higher the steerability.

There is a very interesting consequence of this construction. Namely, we can explicitly find this smallest circle. This way we can explicitly state what is the largest curvature that the robot can follow as throughout following this circle there is no need to update the relative position of the cart.

Construction of this circle goes as follows. The circle needs to pass through the base, with the base's heading coinciding with the circle's tangent at that point. Set the center of circle at the base's position, making it a point. Now move the circle's center along base frame's x axis. This increases the radius r . Keep moving the center until the circle intersects the workspace — don't forget that the circle's tangent needs to be possible for a given (x, y) in the workspace.

Additional consideration The cart needs to be of the same distance from the base of the robot due to some other calculations in the path following control strategy. The notion of distance does not matter too much. So we choose Euclidean distance to make life simpler. This corresponds to a semicircle in the workspace. Thus the task is to find the best radius for steerability. On one hand, we want the smallest possible radius, as discussed above. On the other hand, we need the most available heading. However, due to the dual arms, selecting a point close to the base, and a heading for the arms different then the heading of the base, results in self-collisions of the end-effectors in the base. As in the target point is literally in the base. There is a band where all headings are available. So we select the smallest radius with all available headings. And the task is to find it. And for this we need to have the workspace. But, for the purposes of having a nice plot, we search a whole rectangle. Hence the exercise of finding the workspace is a bit more general.

2 Finding the workspace

Following a path to get to a point is more difficult than just reaching a point. I have a whole paper idea for how to address this question, but as it stands now this is the case. Thus every point should be reached from a known comfortable position.

A point is given by $p = [x \ y \ \theta]$. For a single armed robot, θ is pointless as they usually the last joint is revolute and can match any heading (provided no added restrictions). But for a dual armed robot, this actually moves whole arms and can result in self-collisions. Of course, the specific fixed end-effector rotation R will change things. We have end-effector z axis being antiparallel to the world z axis.

So we generate a list of coordinates $\mathbf{x}, \mathbf{y}, \boldsymbol{\theta}$. For every combination of these, we get an SE3 target:

$$T_w^{goal} = \begin{bmatrix} R \cdot R_z(\theta_k) & \begin{bmatrix} x_i \\ y_j \\ h \end{bmatrix} \\ 0 & 1 \end{bmatrix} \quad (1)$$

Thus the algorithm for finding the workspace has to be $O(n^3)$. Given this, we can find left and right end-effector targets:

$$T_w^{l,r} = T_{goal}^{l,r} T_w^{goal} \quad (2)$$

These can be immediately checked for self-collision without even running IK. In the specific setting we have 1, we actually also need to check for the collision of the cart and the base. The cart end-points are determined directly from T_w^{goal} via T_{goal}^i for each corner i . The base is approximated as a rectangle so the distance check is super easy - just check each coordinate, and if they're all in their range, you're in the rectangle. Again, distance the invalid T_w^{goal} immediately.

In total, we have the following.

Algorithm 1 Workspace finding algorithm

```

1: procedure FINDWORKSPACE( $\mathbf{x}, \mathbf{y}, \boldsymbol{\theta}$ )
2:   for  $(i, j, k) \in \mathbf{x} \times \mathbf{y} \times \boldsymbol{\theta}$  do
3:      $q \leftarrow q_{home}$ 
4:      $\mathcal{W} \leftarrow \mathbf{0} \in \mathbb{R}^{N_x \times N_y \times N_\theta}$ 
5:      $T_w^{goal} \leftarrow \begin{bmatrix} R \cdot R_z(\theta_k) & \begin{bmatrix} x_i \\ y_j \\ h \end{bmatrix} \\ 0 & 1 \end{bmatrix}$ 
6:      $T_w^l \leftarrow T_w^{goal} T_{goal}^l$ 
7:      $T_w^r \leftarrow T_w^{goal} T_{goal}^r$ 
8:     if selfCollision(baseShape,  $T_w^l, T_w^r$ , cartCorners( $T_w^{goal}$ )) then
9:       continue
10:    end if
11:    bool converged = InverseKinematics( $q, T_w^l, T_w^r$ )
12:     $\mathcal{W}_{i,j,k} \leftarrow \text{converged}$ 
13:  end for
14: end procedure

```

The inverse kinematics check can be made more efficient in the following way. If after an update $\|q_i - q_{i+1}\| < \epsilon$, stop trying to go further.

3 Result analysis

We have the whole cube (x, y, θ) as a collection of points. Since this is useless to plot outright, we do the following. We take a circle slice of the plane, and plot the headings. This is a 2D surface in 3D that is a part of a cylinder. On it, a blue point means it was reachable, red means it was not reachable.

TODO: put a few fronts here. This shows how points actually look - there are some weird points here and there.

But we don't care about these weird points. We want the continuous interval of headings $[\theta_{min}, \theta_{max}]$ for every point (x, y) . If there are more of them, choose the one closest to $\theta = 0$ — we don't care about the boundary. TODO: put figure of planar workspace with headings via colors here. The heading range is denoted by color. NOTE: for this first plot, we didn't find the interval, but simply summed the number of available headings.

3.1 Maximum steerability semicircle with center at the base

Now we judge the quality of a semicircle, or rather the cylinder slice.